Day 71: Additive (Bahdanau) Attention, From Scratch

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 11, Day 2 | Complete study guide

Watch: Attention for Neural Networks, Clearly Explained!!! (StatQuest) | Lecture 10: Neural Machine Translation and Models with Attention (Stanford)

Script: `day71_additive_attention.py` · every number in this guide is copied by program from `day71_run_output.txt`, the real output of that script.

Day 70 found that a decoder which sees only the encoder's final state cannot copy 20 digits: exact match was 0.004 with hidden 32 and 0.157 with hidden 64 after 40,000 steps. Today the decoder keeps every encoder state and, at each output step, reads a weighted average of them, choosing the weights itself: **attention**. On exactly Day 70's test, with hidden 32 and half the steps, every seed copies every test sequence of every length correctly (1.000); even hidden 16 does.

0. Where today fits

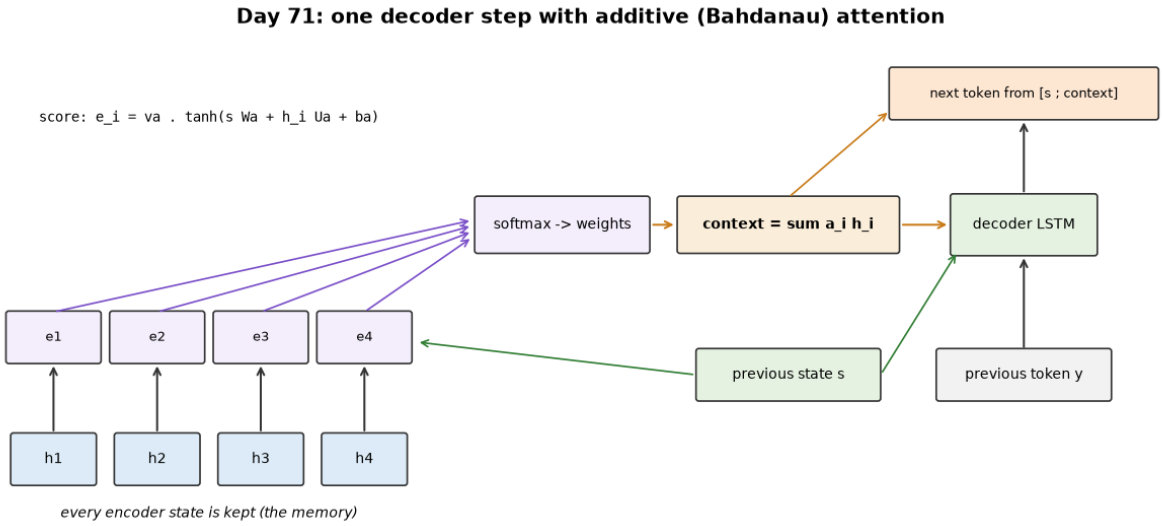
DEFINITION — start here, no prior knowledge assumed

Attention lets the decoder look back at all encoder states $h_1, \ldots, h_n$ at every output step. It scores each one against its own state, turns the scores into weights with a softmax, and reads the weighted average, the **context**. In **additive** attention (Bahdanau et al., 2015) the score is $v_a \cdot \tanh(s\,W_a + h_i\,U_a)$: the two are projected, *added*, squashed and scored.

REAL-WORLD SCENARIO

Translating with the source text open in front of you instead of from memory: for each word you write, you glance back at the part of the source that matters now.

SYSTEM DIAGRAM



One decoder step. Every encoder state is scored against the decoder's previous state; the weights mix the states into a context; the decoder LSTM reads the previous token and the context.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. One attention step, by hand

DEFINITION — start here, no prior knowledge assumed

Three encoder states, one decoder state, small hand-chosen weights: every score, weight and the context are computed with plain arithmetic and compared with the functions.

REAL-WORLD SCENARIO

Doing one exchange-rate conversion on paper before trusting the spreadsheet.

MATHEMATICS

$$k_i = h_i U_a + b_a \quad \textit{(keys, once per sequence);} \quad q = s W_a \quad \textit{(the query)}$$

$$e_i = v_a \cdot \tanh(q + k_i); \quad \alpha = \textit{softmax}(e); \quad \textit{context} = \Sigma_i \alpha_i h_i$$

SPECIFICATION

Setup h = (1, 0), (0, 1), (1, 1); s = (1, –1); W_a = identity, U_a = 0.5 · identity, b_a = 0, v_a = (1, –1).

Why this test

The score function is the one new formula; checking it by hand first makes the gradient check easier to trust.

IMPLEMENTATION

```
def additive_scores(s_prev, keys, Wa, va):
    t = np.tanh((s_prev @ Wa)[:, None, :] + keys)
    return t @ va, t
```

ACTUAL OUTPUT

```
encoder state   key           tanh(query + key)       score by hand  function   weight
[1.0, 0.0]      [0.5, 0.0]    [0.9051, -0.7616]              1.6667    1.6667   0.4196
[0.0, 1.0]      [0.0, 0.5]    [0.7616, -0.4621]              1.2237    1.2237   0.2694
[1.0, 1.0]      [0.5, 0.5]    [0.9051, -0.4621]              1.3673    1.3673   0.3110
weights by hand [0.4196, 0.2694, 0.311]; context = sum of weight x state = [0.7306, 0.5804] (by
    hand [0.7306, 0.5804])
Actual output from this lesson's run.
```

What it means

The function matches the hand arithmetic on every score. The decoder state (1, –1) prefers the first encoder state, (1, 0), which gets the largest weight (0.4196); the context is the weighted average of the three states.

WHY CHOOSE THIS?

A tiny case where every number can be followed.

WHY NOT THIS? (TRADEOFFS)

Hand-chosen weights; training chooses very different ones.

2. Gradient check: attention, decoder and encoder

DEFINITION — start here, no prior knowledge assumed

The backward pass now has three new paths: through the context into every encoder state, through the softmax into the scores, and through the scores into the decoder's previous state and the keys. Each decoder step's input depends on its previous state through attention, so the decoder is differentiated one step at a time (`lstm_step_backward`, the body of Day 65's loop).

REAL-WORLD SCENARIO

Checking every connection in a rewired circuit board, not just the new chip.

MATHEMATICS

$$d\,h_i \mathrel{+}= \alpha_i\, d\,context; \quad d\alpha_i = d\,context \cdot h_i; \quad de = \alpha \times (d\alpha - \Sigma_i\, \alpha_i\, d\alpha_i)$$

$$d\,s_{prev} \mathrel{+}= (\Sigma_i\, da_i)\, W_a^{\,T}, \quad da_i = de_i\, v_a\, (1 - t_i^2); \quad d\,h_i \mathrel{+}= da_i\, U_a^{\,T}$$

SPECIFICATION

Model	Hidden 4, attention size 3, 3 REVERSE sequences of 5 digits; every entry of all 10 parameter arrays.
Twice	Decoder starting from the encoder's final state (Day 69), and from zeros (attention is then the encoder's only route to the loss).

Why this test

The attention backward pass has more places to go wrong than anything since Day 65.

IMPLEMENTATION

```
dctx += dx[:, V:]
alpha = st["alpha"]
dHs += alpha[:, :, None] * dctx[:, None, :]
dalpha = np.einsum("mh,mnh->mn", dctx, Hs)
de = alpha * (dalpha - (alpha * dalpha).sum(axis=1, keepdims=True))
dsp, dk, dWa, dva = additive_scores_backward(de, st["t"], st["s_prev"], P["Wa"], P["va"])
g["Wa"] += dWa; g["va"] += dva; dkeys += dk
ds, dc = ds_prev + dsp, dc_prev
```

ACTUAL OUTPUT

```
decoder start   parameter  entries  max relative error  max |difference|   pass
encoder end            We      224            1.23e-06          3.74e-11   True
encoder end            be       16            1.62e-08          3.29e-11   True
encoder end            Wd      288            3.75e-06          3.93e-11   True
encoder end            bd       16            5.44e-09          2.93e-11   True
encoder end            Wa       12            1.17e-05          2.53e-11   True
encoder end            Ua       12            2.54e-05          2.81e-11   True
```

```
encoder end            ba        3            1.40e-06          1.96e-11   True
encoder end            va        3            1.93e-07          7.40e-12   True
encoder end           Why       80            8.73e-08          3.56e-11   True
encoder end            by       10            1.45e-09          2.31e-11   True
zeros                  We      224            4.09e-06          4.72e-11   True
zeros                  be       16            1.06e-08          2.98e-11   True
zeros                  Wd      288            3.80e-06          3.41e-11   True
zeros                  bd       16            9.87e-09          2.44e-11   True
zeros                  Wa       12            2.06e-05          2.64e-11   True
zeros                  Ua       12            1.53e-05          2.49e-11   True
zeros                  ba        3            6.91e-07          2.27e-11   True
zeros                  va        3            3.79e-07          1.43e-11   True
zeros                 Why       80            6.52e-08          3.19e-11   True
zeros                  by       10            1.22e-09          3.20e-11   True
every entry passes: decoder starts from the encoder True, from zeros True
Actual output from this lesson's run.
```

What it means

Every entry passes in both versions, including the encoder weights when attention is their only route to the loss.

WHY CHOOSE THIS?

Every gradient path is confirmed before any training result is read.

WHY NOT THIS? (TRADEOFFS)

A tiny model; the trained one is larger but runs the same code.

3. Day 70's hardest test, with attention

DEFINITION — start here, no prior knowledge assumed

Exactly Day 70's Section 2 setup: COPY, lengths 3 to 20 in training, the same 300 fixed test sequences per length, plain SGD at learning rate 0.5, clipping at 1.0, three seeds. Only the model differs.

REAL-WORLD SCENARIO

The same exam as yesterday, but now the student may keep the textbook open.

SPECIFICATION

Model	Hidden 32, attention size 32; 20,000 steps (half of Day 70's 40,000); exact match every 2,500 steps.
Reference	Day 70 at 40,000 steps, exact match at 20 digits: hidden 32 0.004, hidden 64 0.157.

Why this test

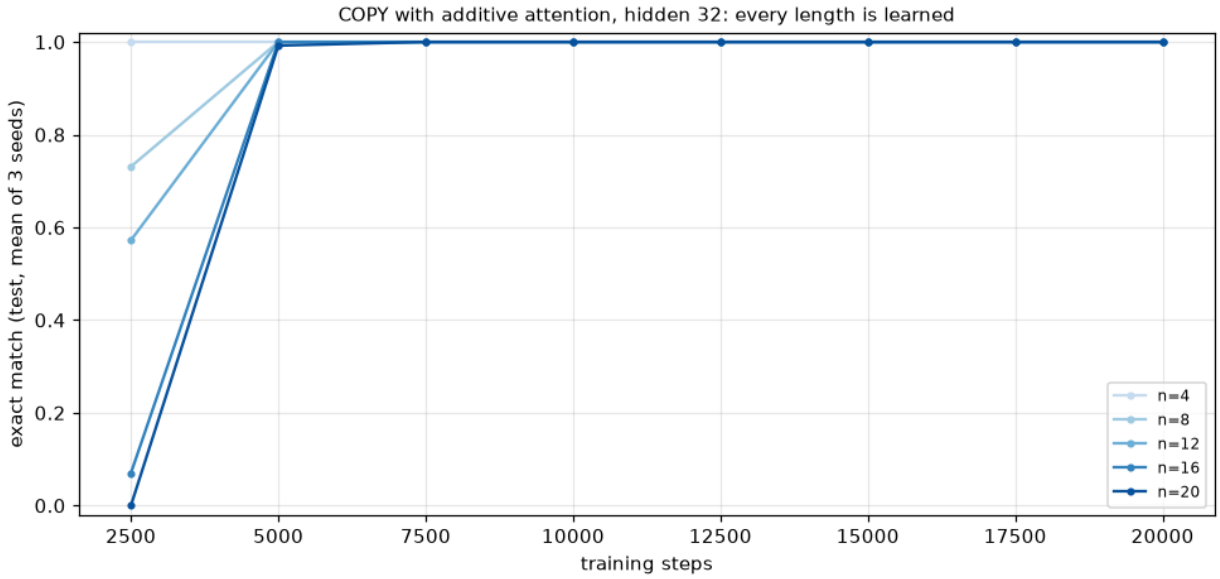
If the bottleneck was the problem, removing it should fix exactly the test that failed.

IMPLEMENTATION

```
def step(self, y_onehot, s, c, Hs, keys):
    e, t = additive_scores(s, keys, self.P["Wa"], self.P["va"])
    ctx, alpha = soft_read(Hs, e, np.ones(e.shape, bool))
    x = np.concatenate([y_onehot, ctx], axis=1)
    s_new, c_new, gates = lstm_step_forward(x, s, c, self.P["Wd"], self.P["bd"])
    out = np.concatenate([s_new, ctx], axis=1)
    return s_new, c_new, dict(s_prev=s, c_prev=c, t=t, alpha=alpha, ctx=ctx, x=x, s=s_new,
        c=c_new, gates=gates, out=out)
```

ACTUAL OUTPUT

```
  steps     n=4     n=8    n=12    n=16    n=20   (mean of 3 seeds)
   2500   1.000   0.731   0.572   0.070   0.000
   5000   1.000   1.000   1.000   1.000   0.992
   7500   1.000   1.000   1.000   1.000   1.000
  10000   1.000   1.000   1.000   1.000   1.000
  12500   1.000   1.000   1.000   1.000   1.000
  15000   1.000   1.000   1.000   1.000   1.000
  17500   1.000   1.000   1.000   1.000   1.000
  20000   1.000   1.000   1.000   1.000   1.000
Actual output from this lesson's run.
```



Exact match during training, every length, mean of three seeds.

What it means

From 7,500 steps on, every seed writes every test sequence of every length exactly right (1.000). After 2,500 steps the longest sequences are not yet learned (0.000 at 20 digits), so attention does not make the task instant; but where Day 70's best model reached 0.157 at 20 digits after 40,000 steps, this one is perfect after 7,500. The bottleneck, not the length of the sequences, was what made Day 70 fail.

WHY CHOOSE THIS?

Removes the bottleneck directly: the decoder reads the input it needs, when it needs it.

WHY NOT THIS? (TRADEOFFS)

Every decoder step now looks at every encoder state: n x n work for n-digit sequences (Section 6).

4. A smaller state, REVERSE, and no hand-over

DEFINITION — start here, no prior knowledge assumed

Three variations, one at a time: a hidden size of 16; the REVERSE task; and a decoder that starts from zeros instead of the encoder's final state, so attention is its only view of the input.

REAL-WORLD SCENARIO

The open-book student again: with a smaller notebook, with the book read backwards, and without being told anything before starting.

SPECIFICATION

Runs	Each variant with seeds 0, 1, 2 and Section 3's settings; final exact match per length.

Why this test

It checks whether the result depends on the state size, the task, or the hand-over Day 69 relied on.

IMPLEMENTATION

```
s, c = (Hs[:, -1], Cs[:, -1]) if self.init_from_encoder else (z, z)
```

ACTUAL OUTPUT

```
model                                  n=4     n=8    n=12    n=16    n=20  weights
copy, hidden 32 (Section 3)          1.000   1.000   1.000   1.000   1.000    17866
copy, hidden 16                      1.000   1.000   1.000   1.000   1.000     5898
reverse, hidden 32                   1.000   1.000   1.000   1.000   1.000    17866
copy, hidden 32, start from zeros    1.000   1.000   1.000   0.999   0.971    17866
Actual output from this lesson's run.
```

What it means

Hidden 16 is enough (1.000 at every length) with 5898 weights, a quarter of the state size of Day 70's largest model, which failed. REVERSE is solved as well. Starting from zeros, attention alone almost matches (0.999 at 16 digits, 0.971 at 20): the hand-over of the final state helps a little at the longest lengths within this budget, but the information arrives mainly through attention.

WHY CHOOSE THIS?

The result holds across state size and task; attention alone carries the input.

WHY NOT THIS? (TRADEOFFS)

Attention adds weights (17,866 against 11,338 for Day 69's model at hidden 32).

5. Longer than any training sequence

DEFINITION — start here, no prior knowledge assumed

The Section 3 models, trained on 3 to 20 digits, tested on 25, 30 and 40 digits.

REAL-WORLD SCENARIO

The open-book student given a longer chapter than any practised on.

SPECIFICATION

Data	300 fixed test sequences per length; three seeds.

Why this test

Attention removed the bottleneck. Does it also let the model handle lengths it has never seen?

IMPLEMENTATION

```
for n in [20, 25, 30, 40]:
    scores = [exact_and_token(train_cached("copy", 32, s)[0], fixed_test(n, "copy")) for s in
        SEEDS]
    e, t = np.mean(scores, axis=0)
```

ACTUAL OUTPUT

```
  length  exact match  token acc          per seed (exact)
      20        1.000      1.000       1.000  1.000  1.000
      25        0.000      0.742       0.000  0.000  0.000
      30        0.000      0.618       0.000  0.000  0.000
      40        0.000      0.465       0.000  0.000  0.000
Actual output from this lesson's run.
```

What it means

No. At 25 digits no sequence is copied exactly (0.000 for every seed), although 0.742 of the digits are still right; at 40 digits 0.465. Attention solves the problem it was built for, a fixed-size summary, but the model still only knows how to find positions it met in training. Day 72 looks at the weights to see where it loses its place.

WHY CHOOSE THIS?

An honest limit: removing the bottleneck is not the same as generalising to any length.

WHY NOT THIS? (TRADEOFFS)

Only one task and one training range were tested.

6. What attention costs

DEFINITION — start here, no prior knowledge assumed

The time of one forward and backward pass on a batch of 32 twenty-digit sequences, for Day 69's model and today's, timed in the same run.

REAL-WORLD SCENARIO

The open book takes desk space and time to consult.

MATHEMATICS

work per decoder step: one score for each of the n encoder states; per sequence: about $n \times n$ scores

IMPLEMENTATION

```
t_plain = best(lambda: plain.loss_and_grads(Xs, Xd, Y))
t_attn = best(lambda: attn.loss_and_grads(Xs, Xd, Y))
```

ACTUAL OUTPUT

```
Day 69 encoder-decoder: 8.3 ms per step, 11338 weights
with additive attention: 20.3 ms per step, 17866 weights; time ratio 2.45
Actual output from this lesson's run.
```

What it means

A training step with attention took 2.45 times as long as Day 69's in this run, with 17866 weights against 11338. The other runs were going on at the same time on this machine, so only the ratio is meaningful, and it would grow with longer sequences.

WHY CHOOSE THIS?

A few times the cost per step for a model that actually solves the task.

WHY NOT THIS? (TRADEOFFS)

The n x n cost becomes the main expense for long inputs; the Transformer (Week 12) has the same property.

Interview preparation — technical questions

Q: Write additive attention.

A: e_i = v . tanh(W s + U h_i + b); alpha = softmax(e); context = sum alpha_i h_i. The decoder reads [y ; context].

Q: Which decoder state does Bahdanau attention use as the query?

A: The previous one, s_{j-1}, so the context is ready before step j's LSTM update.

Q: What does the backward pass of the weighted sum send back?

A: alpha_i times d context to each h_i, and d context . h_i to each weight, then through the softmax.

Q: Why compute the keys once per sequence?

A: h_i U is the same for every decoder step; only the query changes.

Q: Does the decoder still need the encoder's final state?

A: Not much here: starting from zeros it reached 0.999 at 16 digits and 0.971 at 20.

Interview preparation — theoretical questions

Q: What problem does attention solve?

A: The information bottleneck: the decoder no longer has to get everything from one fixed-size state.

Q: What does attention not solve, according to today's results?

A: Generalising to longer inputs than in training: exact match was 0.000 at 25 digits.

Q: Why is it called additive?

A: The query and key are combined by addition inside a tanh, then scored; Day 73's dot-product score multiplies them instead.

Q: What is the cost of attention?

A: One score per encoder state per decoder step: work grows with input length times output length.

Key takeaway and what's next

KEY TAKEAWAY

Keeping every encoder state and letting the decoder choose what to read removes Day 70's bottleneck completely on its own test: perfect copies of 3 to 20 digits at hidden 32 and even hidden 16, where the plain model managed 0.157 at 20 digits with hidden 64. The gradient through attention checks out exactly. What attention does not do is let the model handle lengths it has never seen: at 25 digits every sequence fails.

- Additive score: e_i = va . tanh(s Wa + h_i Ua + ba); weights = softmax; context = weighted sum.
- Decoder input [y ; context]; output from [s ; context].
- Day 70's test with attention: 1.000 exact match at every length, from 7,500 steps.
- Hidden 16 enough; REVERSE too; attention alone (no hand-over) 0.971 at 20 digits.
- Longer than training: 0.000 exact match at 25 digits.

- Cost: 2.45 times a plain step in this run; about n x n scores per sequence.

Suggested watch order, if starting from zero

- 1. Attention for Neural Networks, Clearly Explained!!! (StatQuest) — attention explained step by step.
- 2. Lecture 10: Neural Machine Translation and Models with Attention (Stanford) — attention in neural machine translation, in depth.

Day 72 preview

- **Looking inside attention.** Every decoder step produced a set of weights. Day 72 draws them as alignment heatmaps, measures whether the model looks where the digit is, and finds where it loses its place on long inputs.
- Today's exercise script, `day71_exercise.py`, isolates one additive attention read, forward and backward, gradient-checked.